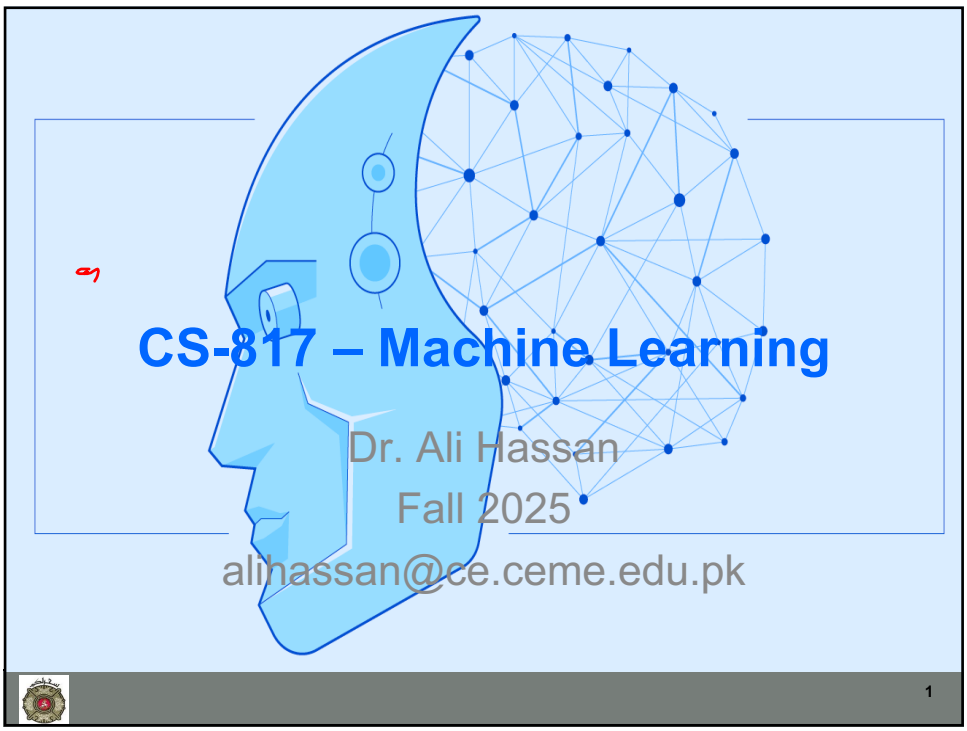
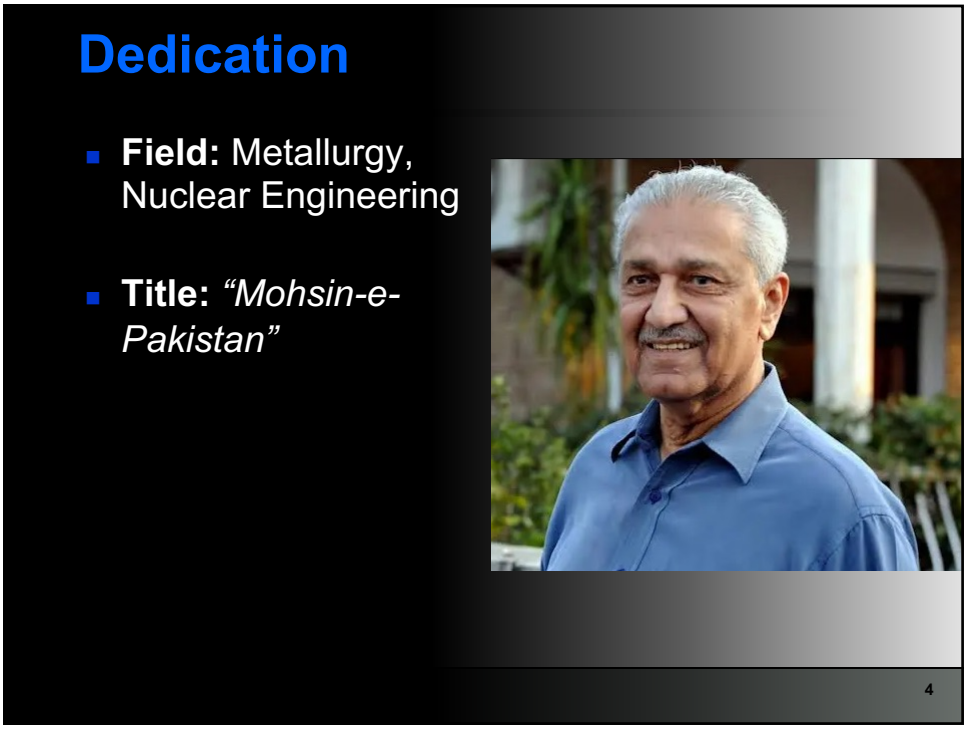

CS-817 – Machine Learning
Dr. Ali Hassan
Fall 2025
alihassan@ce.ceme.edu.pk
1

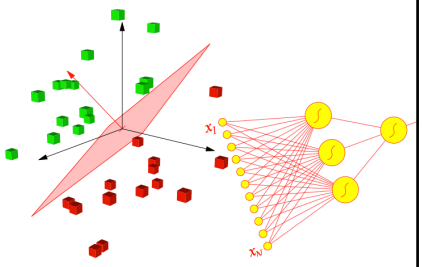
1


Dedication
Field: Metallurgy, Nuclear Engineering
Title: "Mohsin-e-Pakistan"
4

4

Announcements

- Assignments
  - Any Questions



5

5

Outline

- Previous Lecture
  - Linear Regression
    - Uni-Variate
    - Multi-Variate
  - Polynomial Regression
  - Normal Equations
  - Logistic Regression

6

6

Todays Lecture

- This Lecture
  - Logistic Regression
  - Regularized Logistic/Regression

7

7

Classification

LOGISTIC REGRESSION

8

8

Classification

- Email: Spam / Not Spam?
- Online Transactions:
  - Fraudulent (Yes / No)?
- Should a bank give a loan to person or NOT
- Which people are more likely to vote
- Which customers are more likely to buy a new product

9

9

Classification

- Should a student be admitted to a school or not
- Whether people will subscribe to a magazine
- Tumor: Malignant / Benign ?

$y \in \{0, 1\}$

0: "Negative Class" (e.g., benign tumor)
1: "Positive Class" (e.g., malignant tumor)

Supervise Learning: we know the correct answers
Classification: the outputs are discrete

10

10

Logistic Regression

- Unlike Linear Regression, the dependent variable **(y)** can take a limited number of values only i.e, the dependent variable is **categorical**
- When the number of possible outcomes is only two it is called **Binary Logistic Regression**.

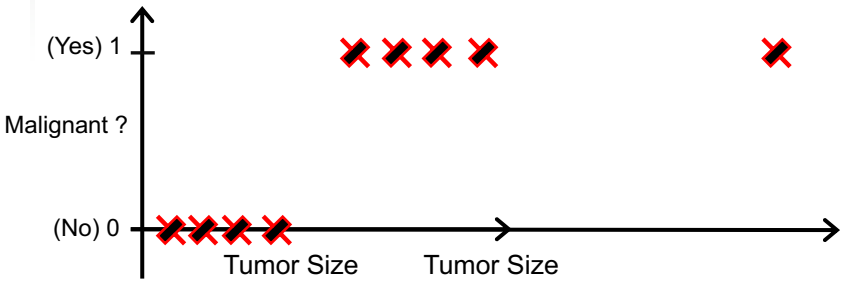
11

11

Threshold classifier output $h_\theta(x)$ at 0.5:

If $h_\theta(x) \geq 0.5$, predict "y = 1"

If $h_\theta(x) < 0.5$, predict "y = 0"



12

12

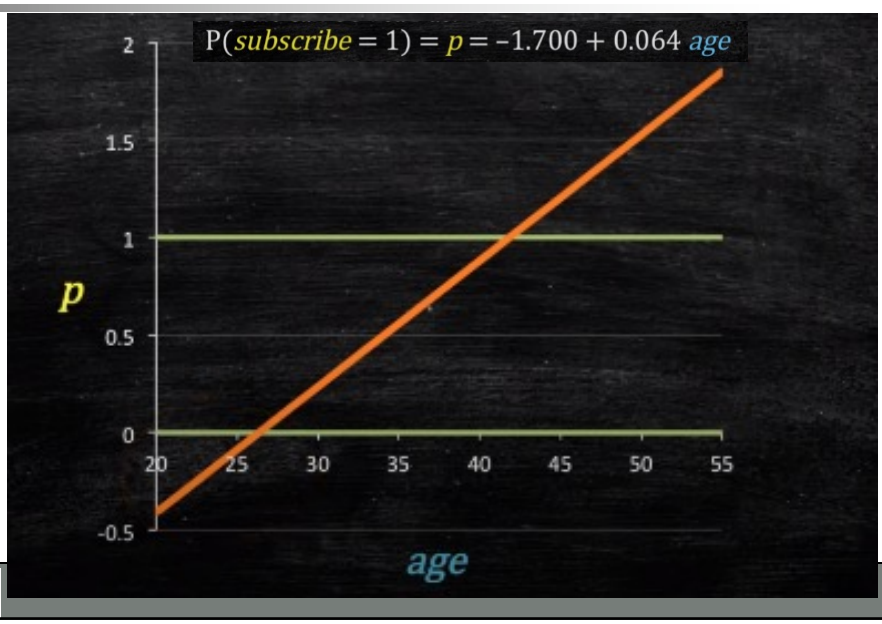
Interpreting this scenario

- If our dependent variable is binary, we want it between 0 and 1
- This can be interpreted as what increases the likelihood of subscription or P(*subscribe=1*)
- Remember, probabilities are bounded by $0 \leq p \leq 1$

13

13

Using Linear Model



14

Restriction on Logistic Reg

- If we take the weighted sum of inputs as the output as we do in Linear Regression, the value can be more than 1
- we want a value between 0 and 1
- we don't output the weighted sum of inputs directly
- we pass it through a function that can map any real value between 0 and 1

15

15

Restriction on LogReg

Classification: y = 0 or 1

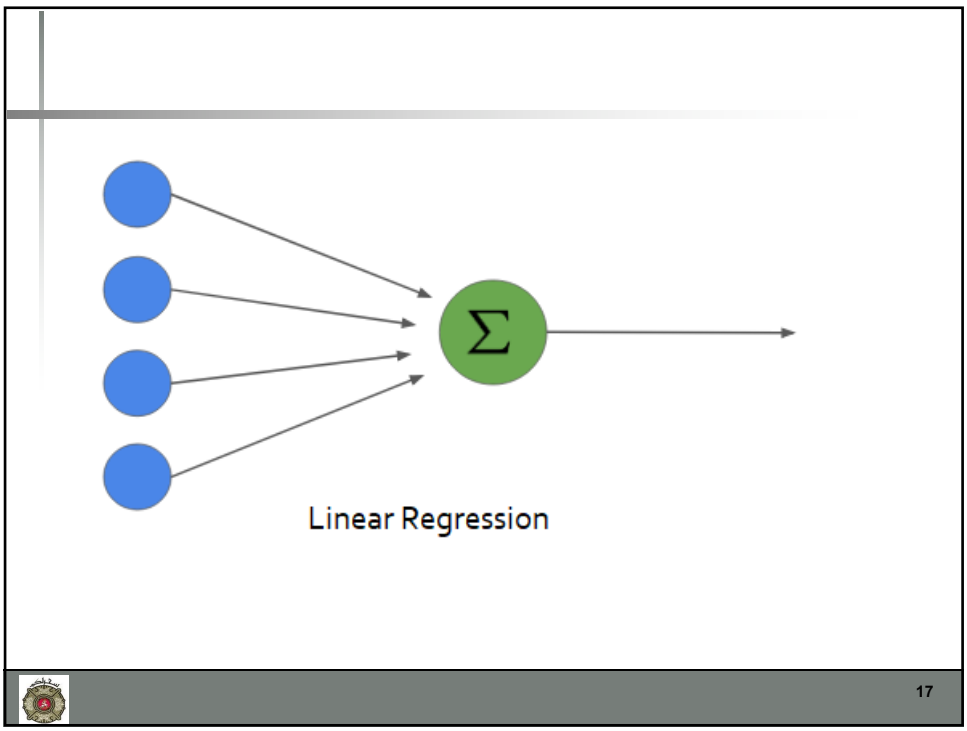
For linear regression:

$h_\theta(x)$ can be > 1 or < 0

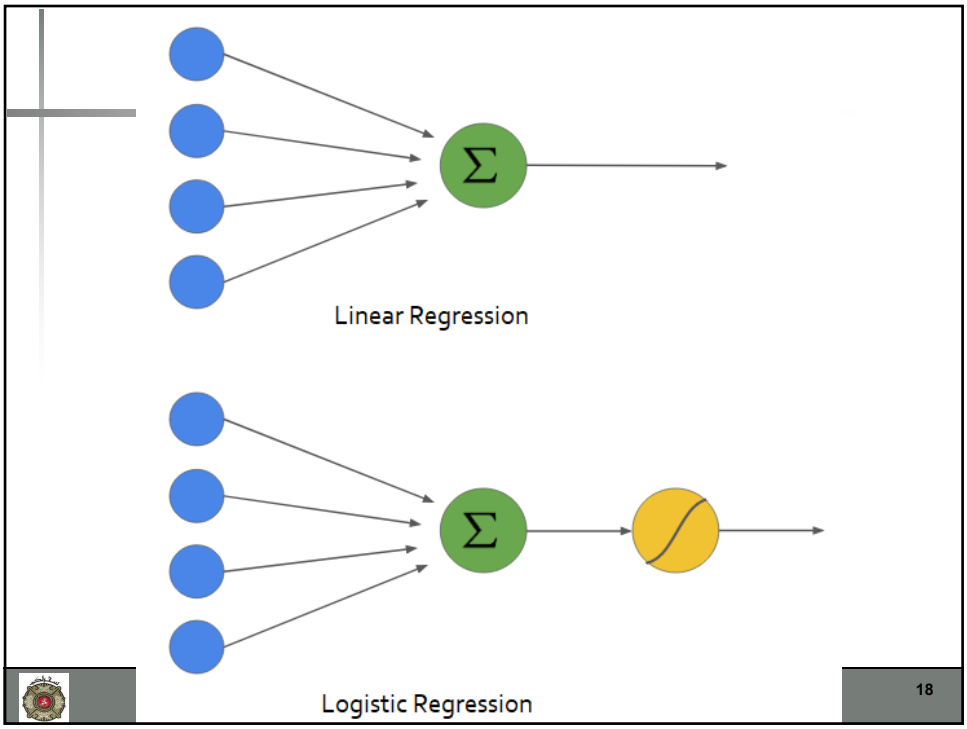
Logistic Regression: $0 \leq h_\theta(x) \leq 1$

16

16


Linear Regression
17

17


Linear Regression
Logistic Regression
18

18

ACTIVATION FUNCTIONS

19

19

Activation Function

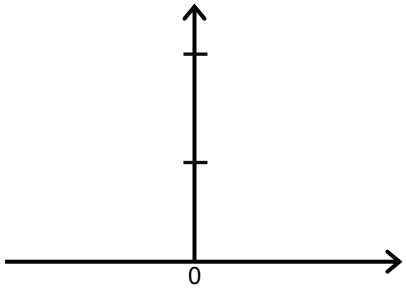
- $h_\theta(x) = \theta^T x$
- Pass this output through a limiting function called Activation function
- $g\left(h_\theta(x)\right) = g(\theta^T x)$
  - Where g is the activation function

20

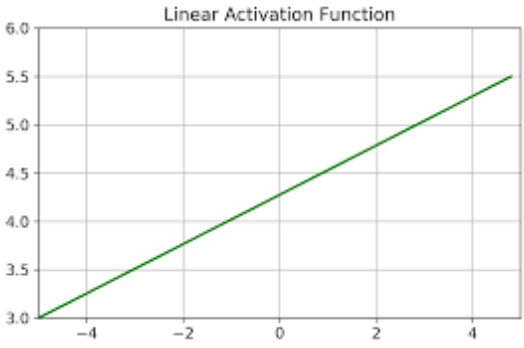
20

Linear Activation Function

- $g(\theta^T x) = \theta^T x$
- No change to the output



21



22

Linear Activation Function

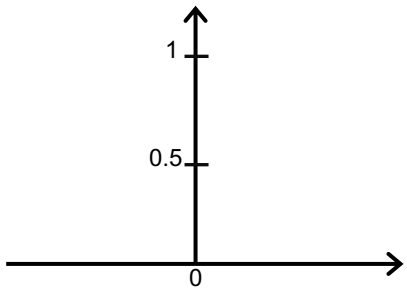
- Similar to the equation of a straight line
- Gives a range of activations from -inf to +inf
- Best suited to for simple regression problems, maybe housing price prediction
- Demerits
  - The derivative of the linear function is the constant

23

23

Sigmoid Activation

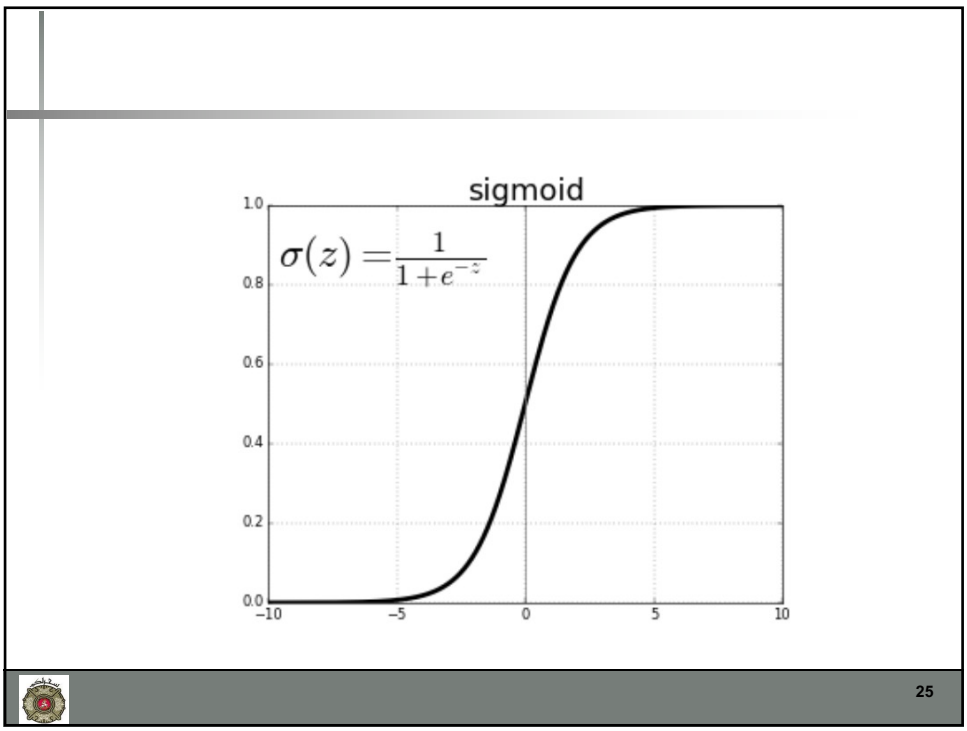
- $g(\theta^T x) = sigm(\theta^T x)$
- $= \frac{1}{1+e^{(-\theta^T x)}}$
- $= \frac{1}{1+e^{-z}}$



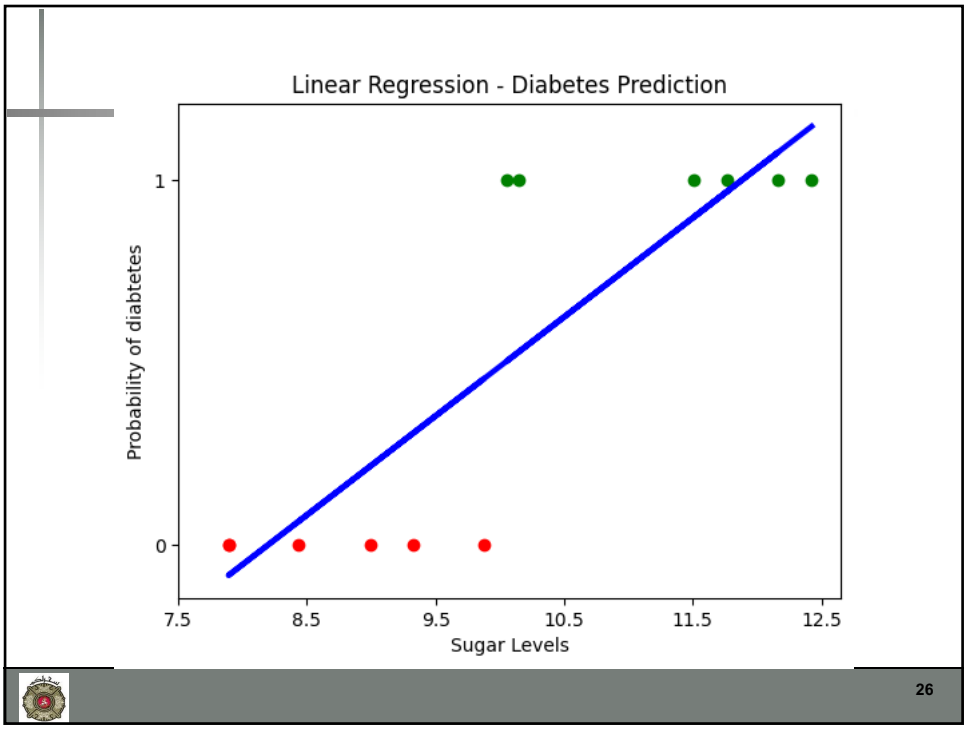
- Where $z = h_\theta = \theta_0 + \theta_1 x_1$
- e is the base of the natural logarithm, approximately equal to 2.718

24

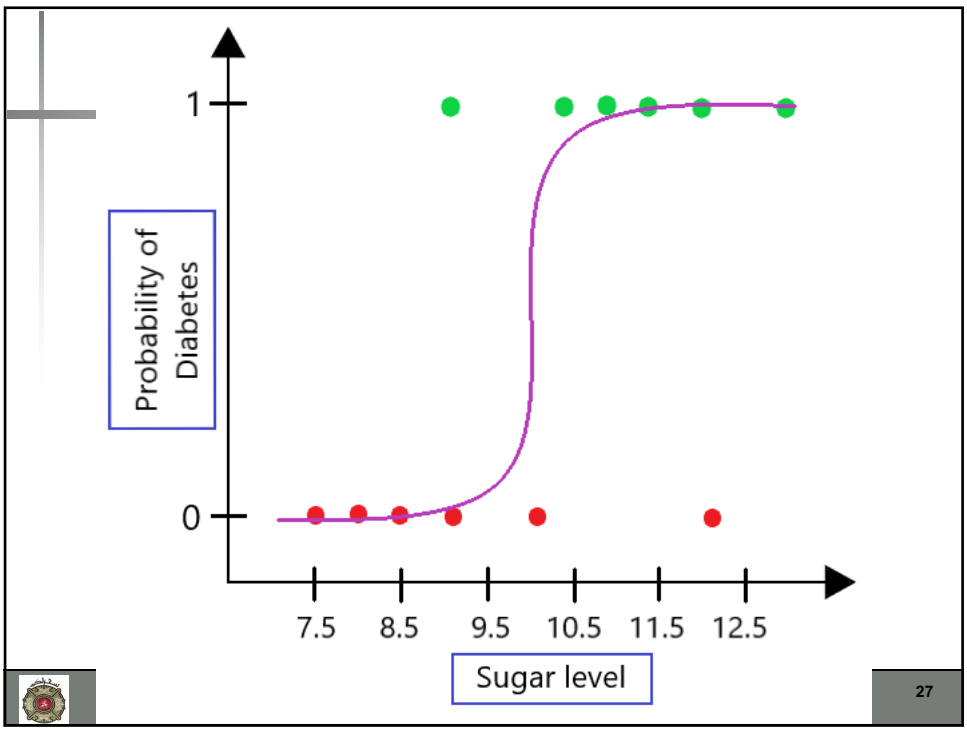
24


sigmoid
$\sigma(z) = \frac{1}{1+e^{-z}}$
1.0
0.8
0.6
0.4
0.2
0.0
−10
−5
0
5
10
25

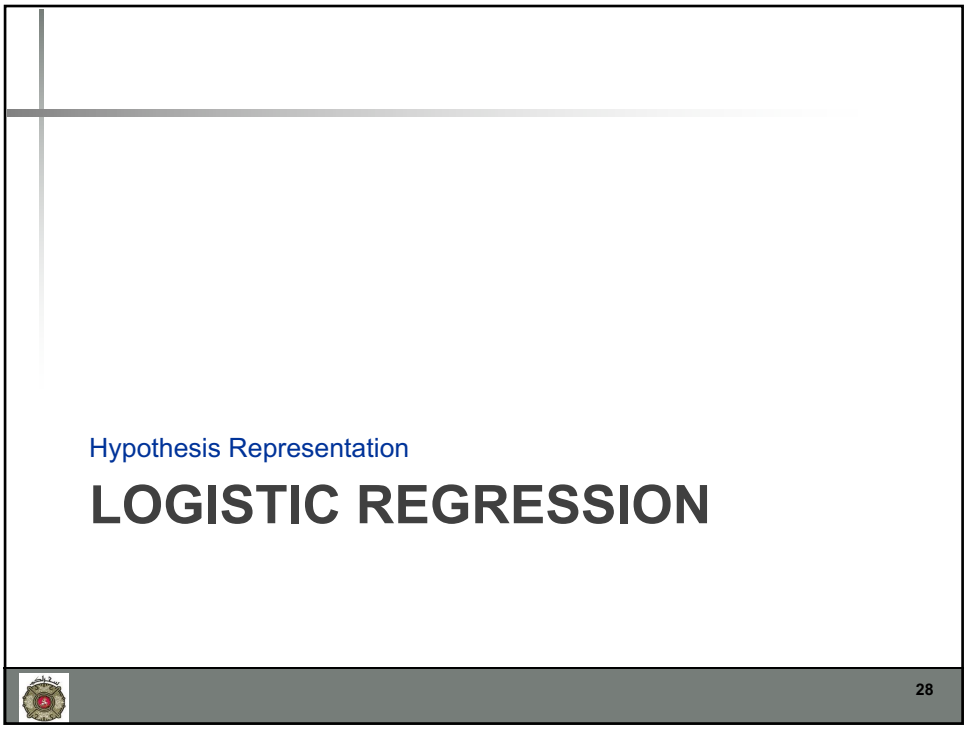
25


Linear Regression - Diabetes Prediction
Probability of diabtetes
1
0
7.5
8.5
9.5
10.5
11.5
12.5
Sugar Levels
26

26


1
0
Probability of Diabetes
7.5
8.5
9.5
10.5
11.5
12.5
Sugar level
27

27


Hypothesis Representation
LOGISTIC REGRESSION
28

28

Logistic Regression Model

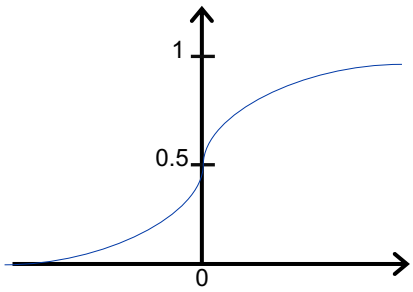
Want $0 \leq h_\theta(x) \leq 1$

$$h_\theta(x) = \theta^T x$$

$$h_\theta(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1+e^{-z}}$$

Sigmoid function
Logistic function



29

29

Sigmoid Func in Python

```
def sigmoid(Z):
    return 1/(1+np.exp(-Z))
```

30

30

Interpretation of Hypothesis Output

$h_\theta(x)$ = estimated probability that y = 1 on input x

Example: If $x = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} 1 \\ \text{tumorSize} \end{bmatrix}$

$$h_\theta(x) = 0.7$$

Tell patient that 70% chance of tumor being malignant

$$h_\theta(x) = P(y = 1 | x; \theta)$$

"probability that y = 1, given x, parameterized by θ "

$$P(y = 0|x;\theta) + P(y = 1|x;\theta) = 1$$
$$P(y = 0|x;\theta) = 1 - P(y = 1|x;\theta)$$

31

31

Decision Boundary

LOGISTIC REGRESSION

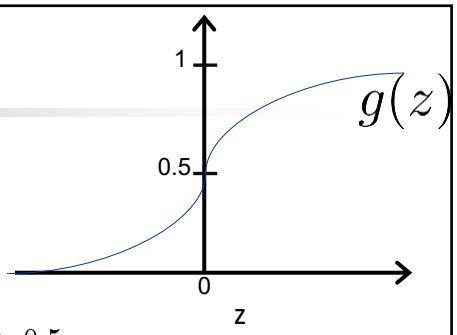
32

32

Logistic regression

$$h_\theta(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1+e^{-z}}$$



Suppose predict "$y = 1$" if $h_\theta(x) \geq 0.5$

$\theta^T x \geq 0$

g(z) ≥ 0.5
When z ≥ 0

predict "$y = 0$" if $h_\theta(x) < 0.5$

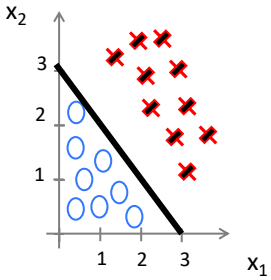
$\theta^T x < 0$

and
g(x) < 0.5
When z< 0

33

33

Decision Boundary



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$$

$$\theta = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$$

Predict: y=1 if $\theta^T x \geq 0$

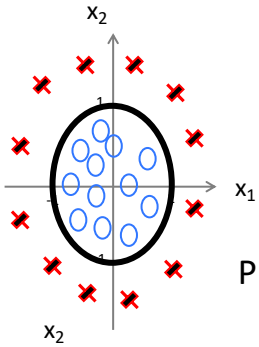
$$-3 + x_1 + x_2 \geq 0$$

34

34

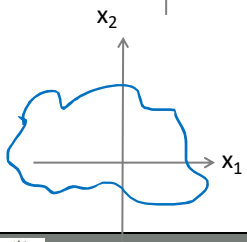
Non-linear decision boundaries

$$\theta = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}$$



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2)$$

Predict "$y = 1$" if $-1 + x_1^2 + x_2^2 \geq 0$



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^2 x_2^2 + \theta_6 x_1^3 x_2 + \ldots)$$

35

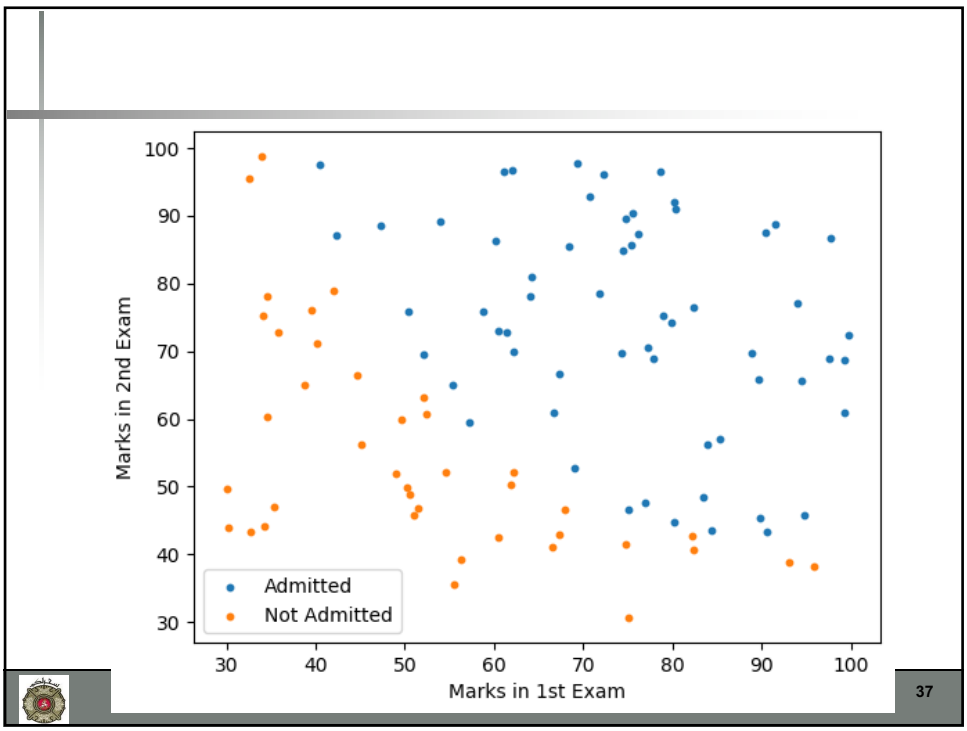
35

Example Data Set

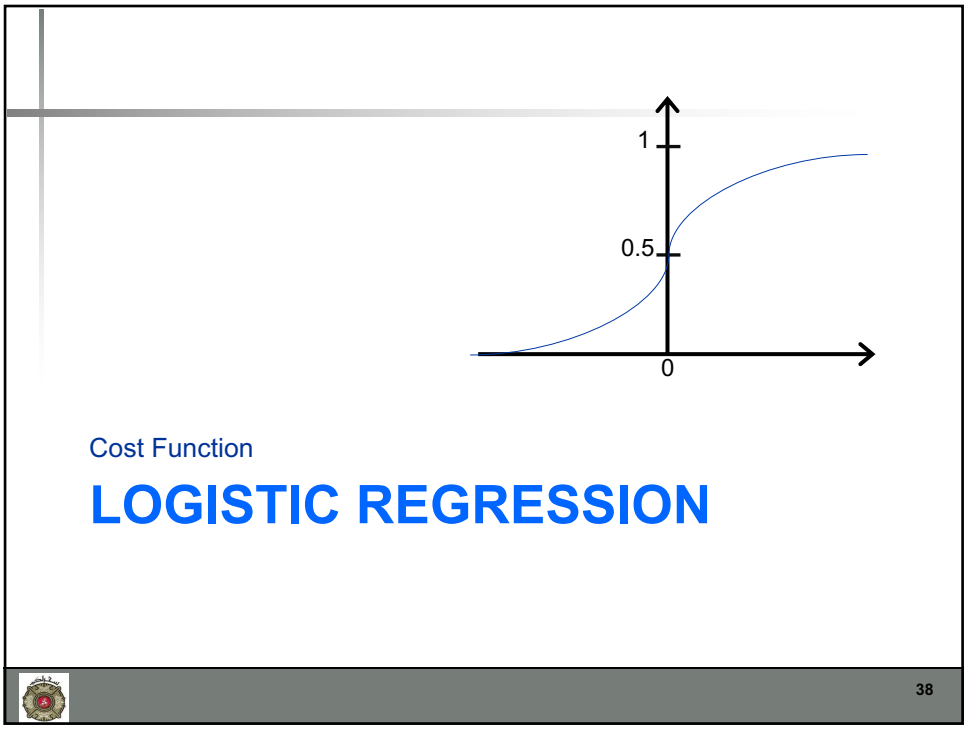
- The data consists of marks of two exams for 100 applicants.
- The target value takes on binary values 1,0.
  - 1 means the applicant was admitted to the university
  - 0 means the applicant didn't get an admission.

36

36


Marks in 2nd Exam
Marks in 1st Exam
Admitted
Not Admitted
100
90
80
70
60
50
40
30
30
40
50
60
70
80
90
100
37

37


1
0.5
0
Cost Function
LOGISTIC REGRESSION
38

38

Training set: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \cdots, (x^{(m)}, y^{(m)})\}$

m examples $x \in \begin{bmatrix} x_0 \\ x_1 \\ \cdots \\ x_n \end{bmatrix}$ $x_0 = 1, y \in \{0, 1\}$

$$h_\theta(x) = \frac{1}{1 + e^{-\theta^T x}}$$

How to choose parameters θ ?

39

39

Cost Function

- For Linear Regression, the conventional Cost Function employed is the MSE
- The cost function (J) for m training samples can be written as:

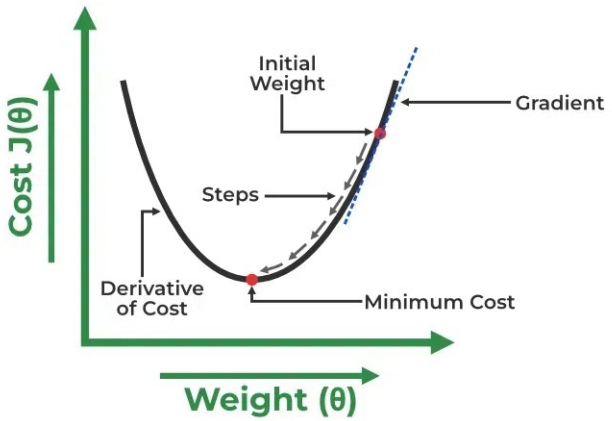
$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} [z^{(i)} - y^{(i)}]^2$$

$$= \frac{1}{2m} \sum_{i=1}^{m} [h_\theta(x^{(i)}) - y^{(i)}]^2$$

40

40

Convex J(θ) for Linear Regression



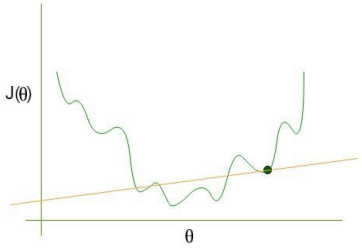
41

41

Why MSE for Logistic Reg

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m}[\sigma^{(i)} - y^{(i)}]^2$$
$$= 1\frac{}{2m\sum_{i=1}^{m}\left(\frac{1}{1+e^{-z^{(i)}}} - y^{(i)}\right)^2}$$
$$= 1\frac{}{2m\sum_{i=1}^{m}\left(\frac{1}{1+e^{-(\theta_1 x^{(i)}+\theta_{\sim 1}}} - y^{(i)}\right)^2}$$

Non Convex Graph for Cost Function



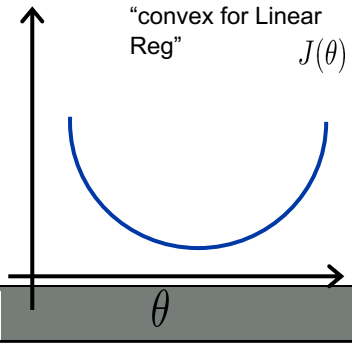
42

Cost function

Linear regression: $J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\frac{1}{2}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$

$\text{Cost}(h_\theta(x^{(i)}), y^{(i)}) = \frac{1}{2}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$

"convex for Linear Reg"



"non-convex for Logistic Reg because of "

$g(z) = \frac{1}{1+e^{-z}}$

43

43

Cost function for Logistic Reg

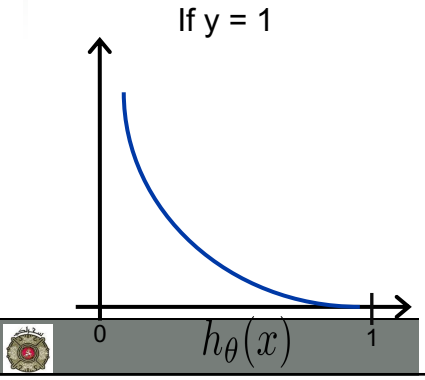
- Log Loss function for Logistic regression

44

44

Logistic regression cost function

$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

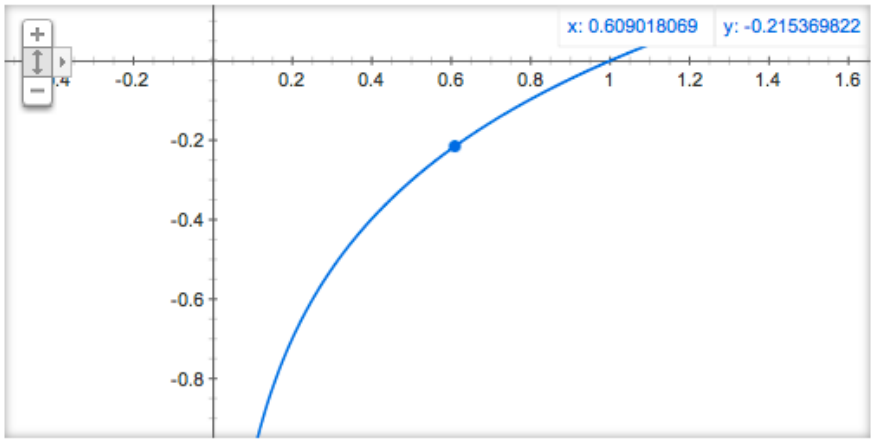


$\text{Cost} = 0$ if $y = 1, h_\theta(x) = 1$
But as $h_\theta(x) \to 0$
$Cost \to \infty$

Captures intuition that if $h_\theta(x) = 0$, (predict $P(y = 1|x; \theta) = 0$), but $y = 1$, we'll penalize learning algorithm by a very large cost.

45

Graph for log(x)



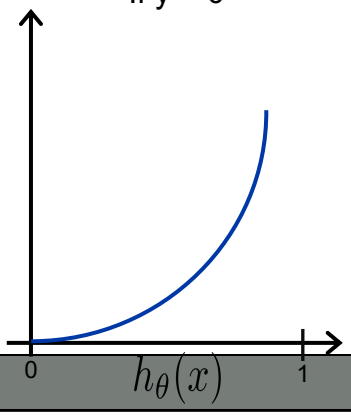
46

46

Logistic regression cost function

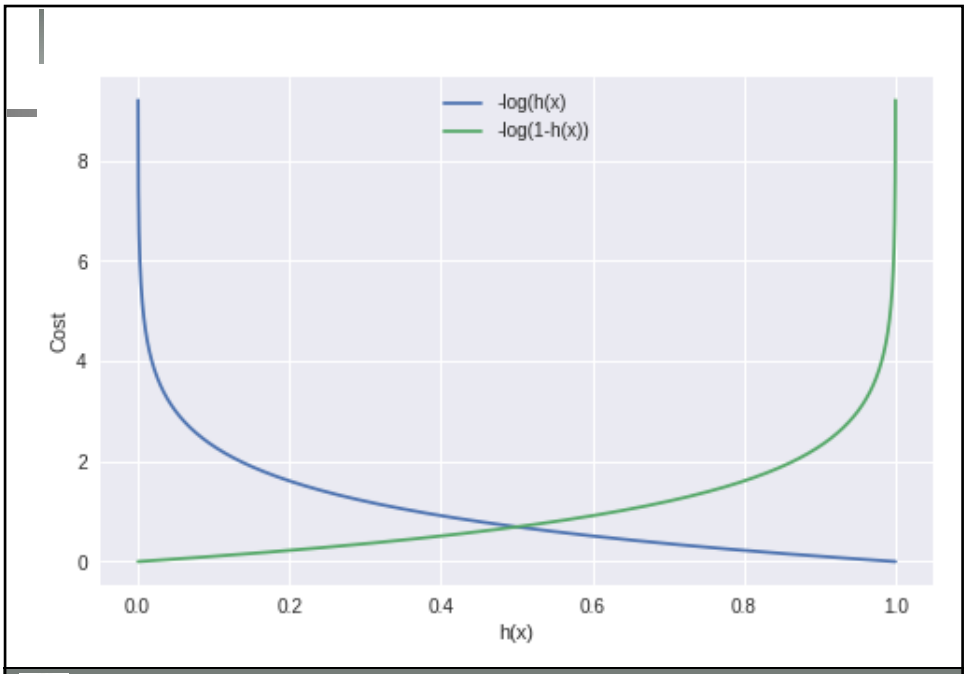
$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

If y = 0



47

47



48

Simplified Cost Function and Gradient Descent

LOGISTIC REGRESSION

49

49

Logistic regression cost function

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$\text{Cost}(h_\theta(x), y) = \begin{cases} -\log(h_\theta(x)) & \text{if } y = 1 \\ -\log(1 - h_\theta(x)) & \text{if } y = 0 \end{cases}$$

Note: $y = 0$ or 1 always

We can combine both of equations using:

$$cost(h(x), y) = -ylog(h(x)) - (1 - y)log(1 - h(x))$$

50

50

Logistic regression cost function

- The cost for all the training examples denoted by *J(θ)* can be computed by taking the average over the cost of all the training samples

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}[y^i log(h(x^i)) + (1-y^i)log(1-h(x^i))]$$

51

51

Logistic regression cost function

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\text{Cost}(h_\theta(x^{(i)}), y^{(i)})$$

$$= -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1-y^{(i)}) \log (1-h_\theta(x^{(i)}))]$$

To fit parameters θ :

$$\min_\theta J(\theta)$$

To make a prediction given new x:

Output $h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$

52

52

Gradient Descent

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

(simultaneously update all θ_j)

$\}$

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^{m} (h(x^i) - y^i)\ x_j^i$$

53

53

Gradient Descent

$$J(\theta) = -\frac{1}{m}[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_\theta(x^{(i)}))]$$

Want $\min_\theta J(\theta)$:

Repeat $\{$

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} (h(x^i) - y^i)\ x_j^i$$

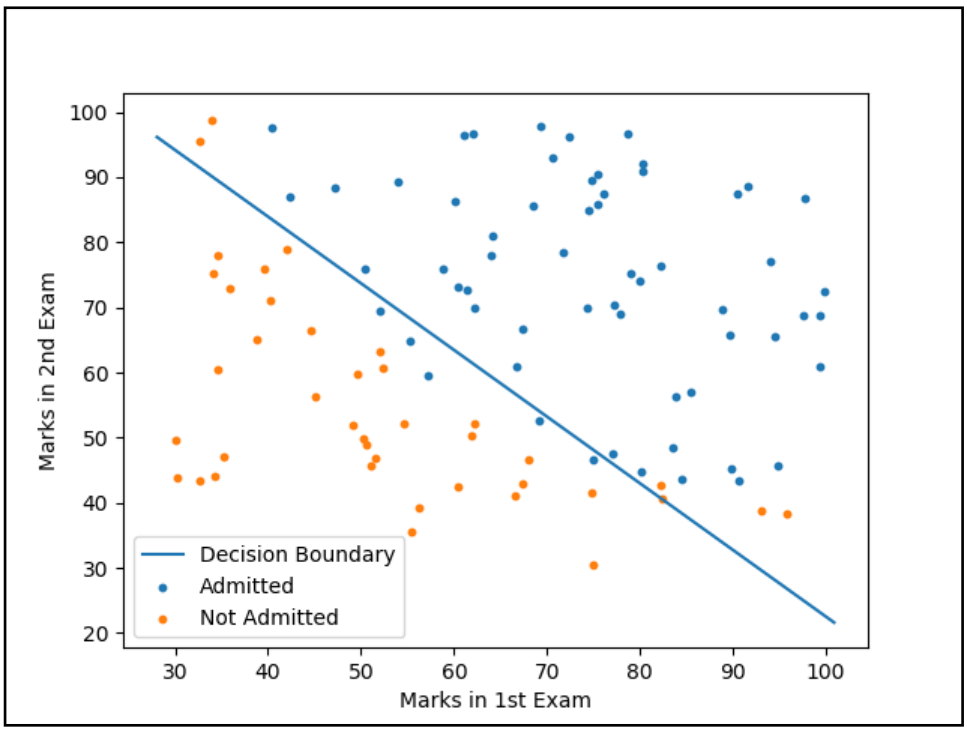
(simultaneously update all θ_j)

$\}$

Algorithm looks identical to linear regression!

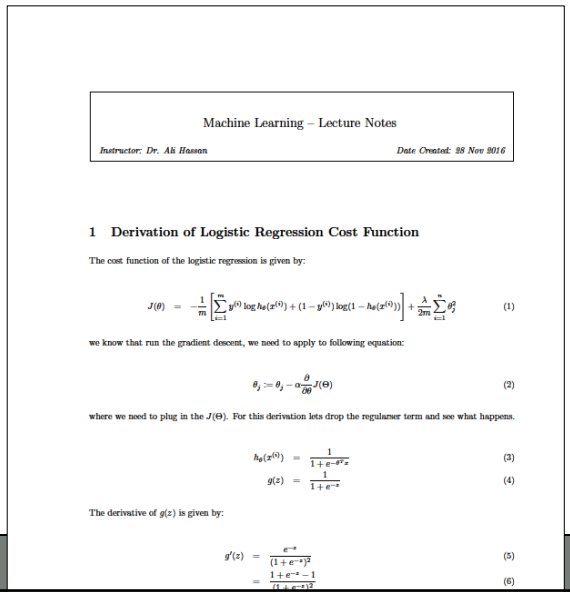
54

54



55

Complete Derivative of Logistic function



Machine Learning – Lecture Notes

Instructor: Dr. Ali Hassan *Date Created: 28 Nov 2016*

1 Derivation of Logistic Regression Cost Function

The cost function of the logistic regression is given by:

$$J(\theta) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} \theta_j^2 \quad (1)$$

we know that run the gradient descent, we need to apply to following equation:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta} J(\Theta) \quad (2)$$

where we need to plug in the $J(\Theta)$. For this derivation lets drop the regularser term and see what happens.

$$h_\theta(x^{(i)}) = \frac{1}{1 + e^{-\theta^T x}} \quad (3)$$

$$g(z) = \frac{1}{1 + e^{-z}} \quad (4)$$

The derivative of $g(z)$ is given by:

$$g'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} \quad (5)$$

$$= \frac{1 + e^{-z} - 1}{(1 + e^{-z})^2} \quad (6)$$

56

56

```
# prompt: implement logistic regression on the diabetes dataset from sklearn.

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_diabetes

# Load the diabetes dataset
diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target

# Convert target variable to binary classification (e.g., > 150)
y_binary = (y > 150).astype(int)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y_binary, test_size=0.25,
random_state=42)

# Create a Logistic Regression model
model = LogisticRegression()

# Train the model
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

57

```
# prompt: implement logistic regression on the diabetes dataset from sklearn using
tensorflow.

import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_diabetes
from sklearn.preprocessing import StandardScaler

# Load the diabetes dataset
diabetes = load_diabetes()
X = diabetes.data
y = diabetes.target

# Convert target variable to binary classification (e.g., > 150)
y_binary = (y > 150).astype(int)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y_binary, test_size=0.2,
random_state=42)

# Scale the data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Define the model
model = tf.keras.Sequential([
tf.keras.layers.Dense(1, activation='sigmoid', input_shape=(X_train.shape[1],))
])

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

58

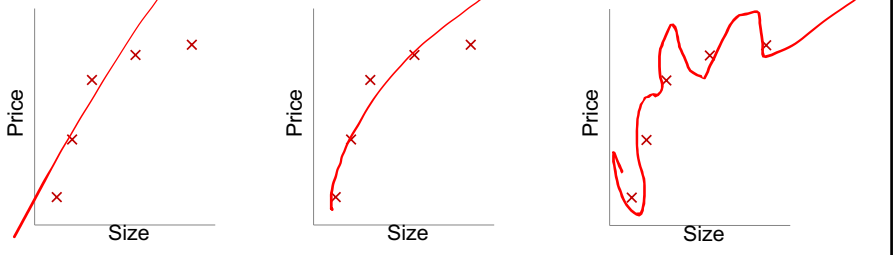
OVERFITTING

59

59

Example: Linear regression (housing prices)

Overfitting: If we have too many features, the learned hypothesis may fit the training set very well ($J(\theta) = \frac{1}{2m}\sum_i (h_\theta(x^{(i)} - y^{(i)})^2 \approx 0$), but fail to generalize to new examples (predict prices on new examples).



$\theta_0 + \theta_1 x$ $\theta_0 + \theta_1 x + \theta_2 x^2$ $\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

60

60

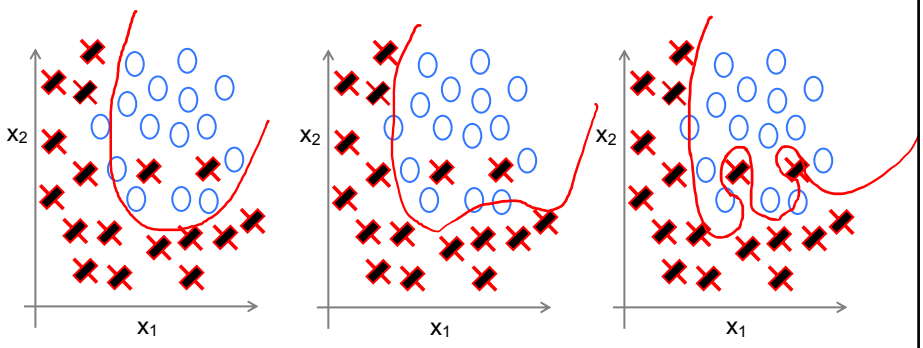
Example: Logistic regression

$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$

(g = sigmoid function)

$g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2)$

$g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \theta_6 x_1^3 x_2 + \ldots)$

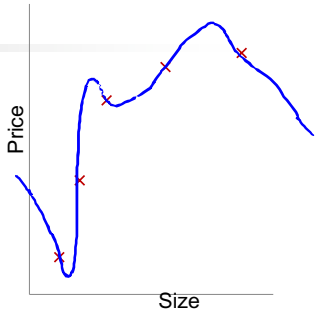


61

61

How to Visualise overfitting:

$x_1 =$ size of house
$x_2 =$ no. of bedrooms
$x_3 =$ no. of floors
$x_4 =$ age of house
$x_5 =$ average income in neighborhood
$x_6 =$ kitchen size
$\vdots$
x_{100}



Plot the function

62

62

Addressing overfitting:

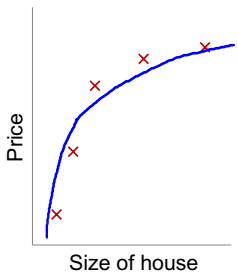
Options:

1. Reduce number of features.
   - Manually select which features to keep.
   - Model selection algorithm (later in course).
2. Regularization.
   - Keep all the features, but reduce magnitude/values of parameters θ_j.
   - Works well when we have a lot of features, each of which contributes a bit to predicting y.

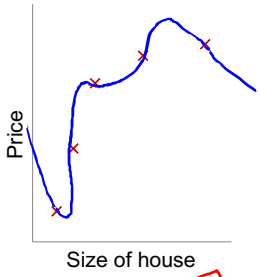
63

63

Intuition



$\theta_0 + \theta_1 x + \theta_2 x^2$



$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$

Suppose we penalize and make θ_3, θ_4 really small.

$$\min_\theta \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda \theta_3 + \lambda \theta_4$$

64

64

Regularization.

Small values for parameters $\theta_0, \theta_1, \ldots, \theta_n$
- "Simpler" hypothesis
- Less prone to overfitting

Housing:
- Features: $x_1, x_2, \ldots, x_{100}$
- Parameters: $\theta_0, \theta_1, \theta_2, \ldots, \theta_{100}$

Regularize

Over Fit Data

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda\theta_j^2\right]$$

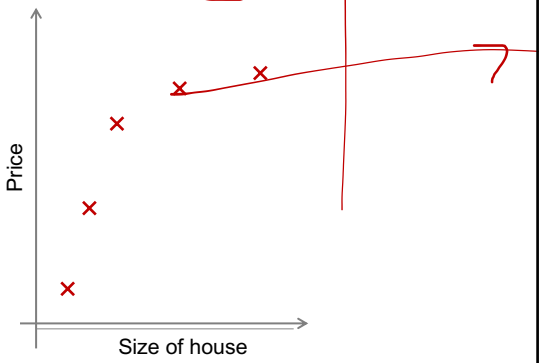
65

Regularization.

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda\sum_{j=1}^{n}\theta_j^2\right]$$

$$\min_\theta J(\theta)$$

- This function has two objective
  - Fitting the data – captured by first term
  - Good generalisation by penalising the parameters – captured by second term



66

In regularized linear regression, we choose θ to minimize

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda\sum_{j=1}^{n}\theta_j^2\right]$$

What if λ is set to an extremely large value (perhaps for too large for our problem, say $\lambda = 10^{10}$)?

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

67

67

Formulation of Cost Function

REGULARISED LINEAR REGRESSION

68

68

Regularized linear regression

$$J(\theta) = \frac{1}{2m}\left[\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^{n}\theta_j^2\right]$$

$$\min_\theta J(\theta)$$

69

69

Gradient descent

Repeat {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_0^{(i)}$$

$$\theta_j := \theta_j - \alpha\left[\frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)} - \frac{\lambda}{m}\theta_j\right]$$

$$(j = 1, 2, 3, \ldots, n)$$

}

$$\theta_j := \theta_j(1 - \alpha\tfrac{\lambda}{m}) - \alpha\frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)}$$

Prev Value − α (derivative)

70

70

Normal equation

$$X = \begin{bmatrix} (x^{(1)})^T \\ \vdots \\ (x^{(m)})^T \end{bmatrix} \qquad y = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

$$\min_{\theta} J(\theta)$$

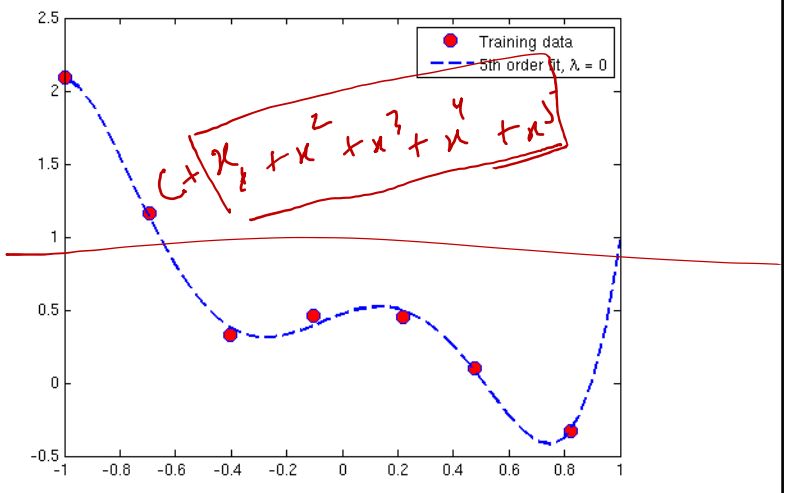
$$\theta = (X^T X)^{-1} X^T y$$

$$\theta = \left(X^T X + \lambda \begin{bmatrix} 0 & 0 \\ 0 & I \end{bmatrix}\right)^{-1} X^T y$$

71

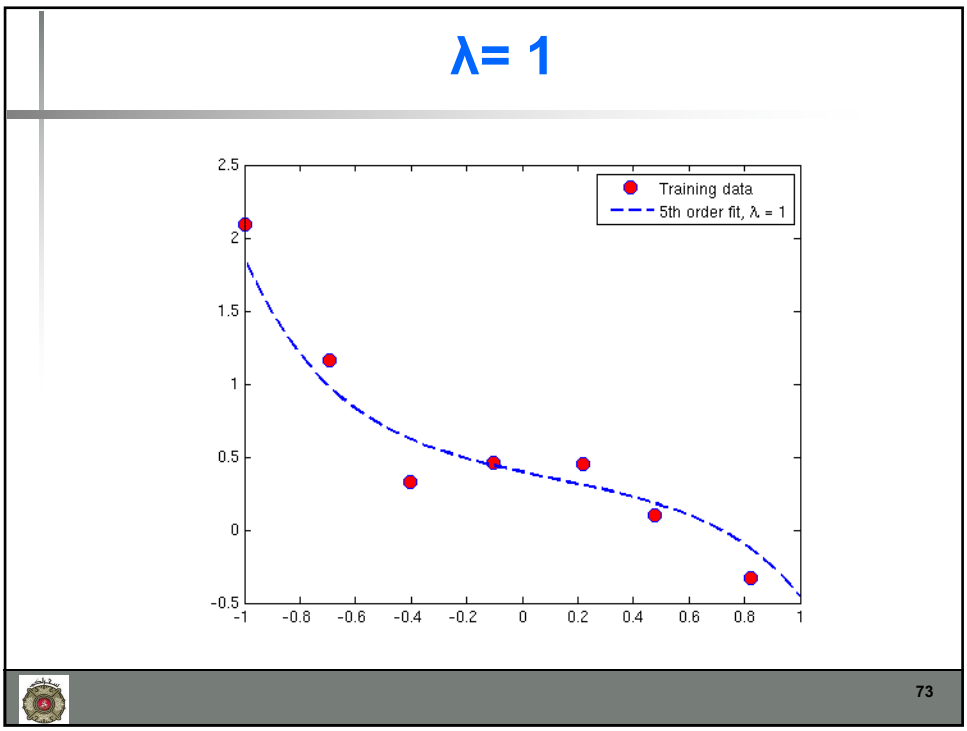
71

Matlab Example: λ=0

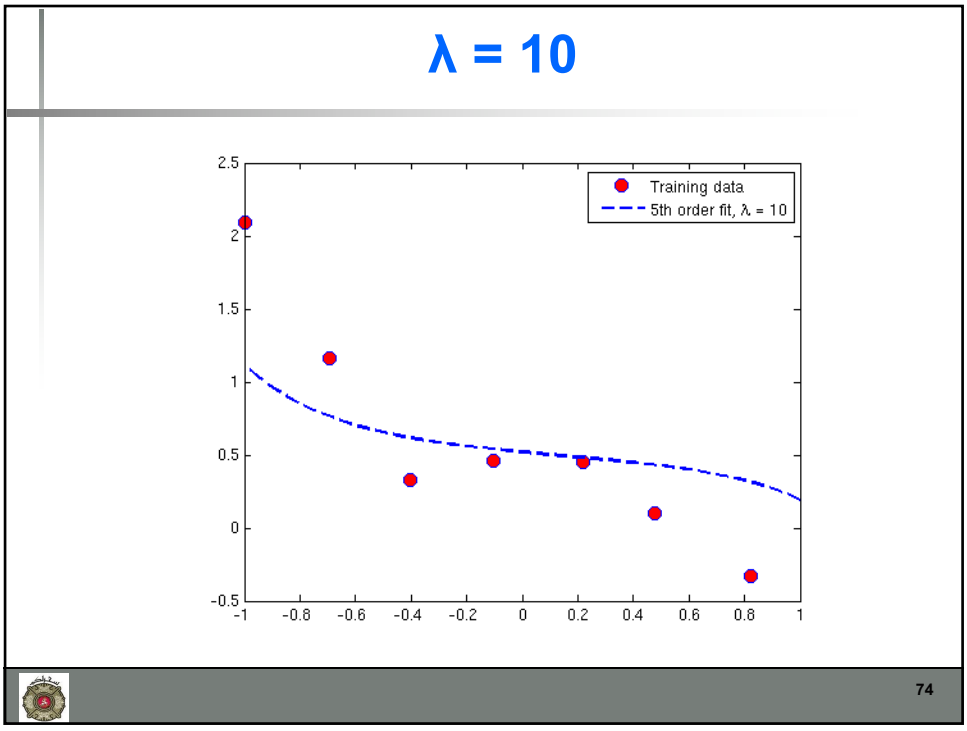


72

72


λ= 1
Training data
5th order fit, λ = 1
2.5
2
1.5
1
0.5
0
-0.5
-1
-0.8
-0.6
-0.4
-0.2
0
0.2
0.4
0.6
0.8
1
73

73


λ = 10
Training data
5th order fit, λ = 10
2.5
2
1.5
1
0.5
0
-0.5
-1
-0.8
-0.6
-0.4
-0.2
0
0.2
0.4
0.6
0.8
1
74

74

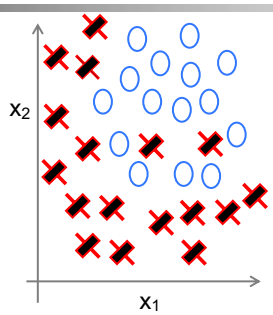
Regularised Logistic Regression

REGULARISATION

75

75

Regularized logistic regression.



$$h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_1^2 + \theta_3 x_1^2 x_2 + \theta_4 x_1^2 x_2^2 + \theta_5 x_1^2 x_2^3 + \dots)$$

Cost function:

$$J(\theta) = -\left[\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} \theta_j^2$$

76

76

Gradient descent

Repeat $\{$

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

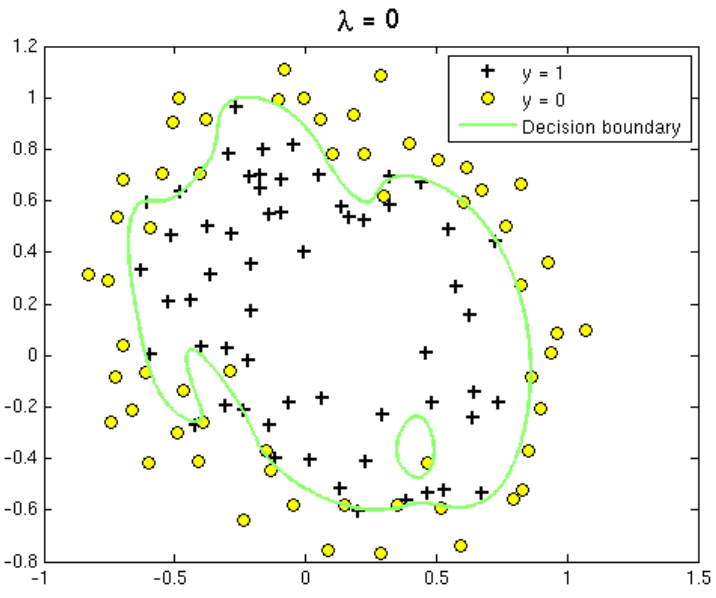
$$\theta_j := \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right]$$

$$(j = 1, 2, 3, \ldots, n)$$

$\}$

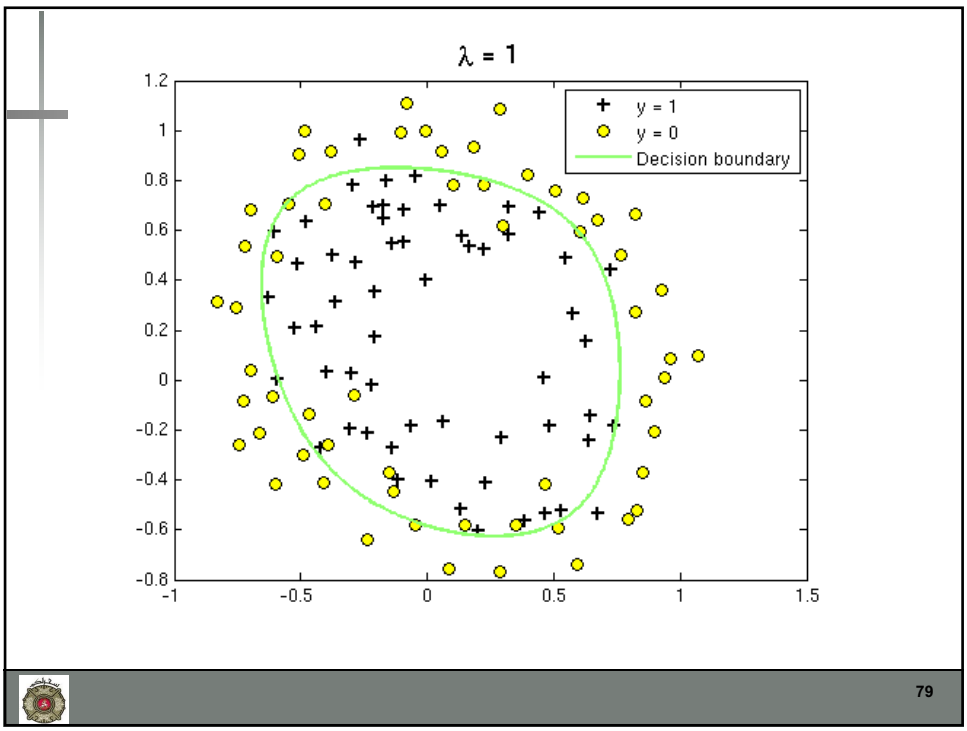
77

77

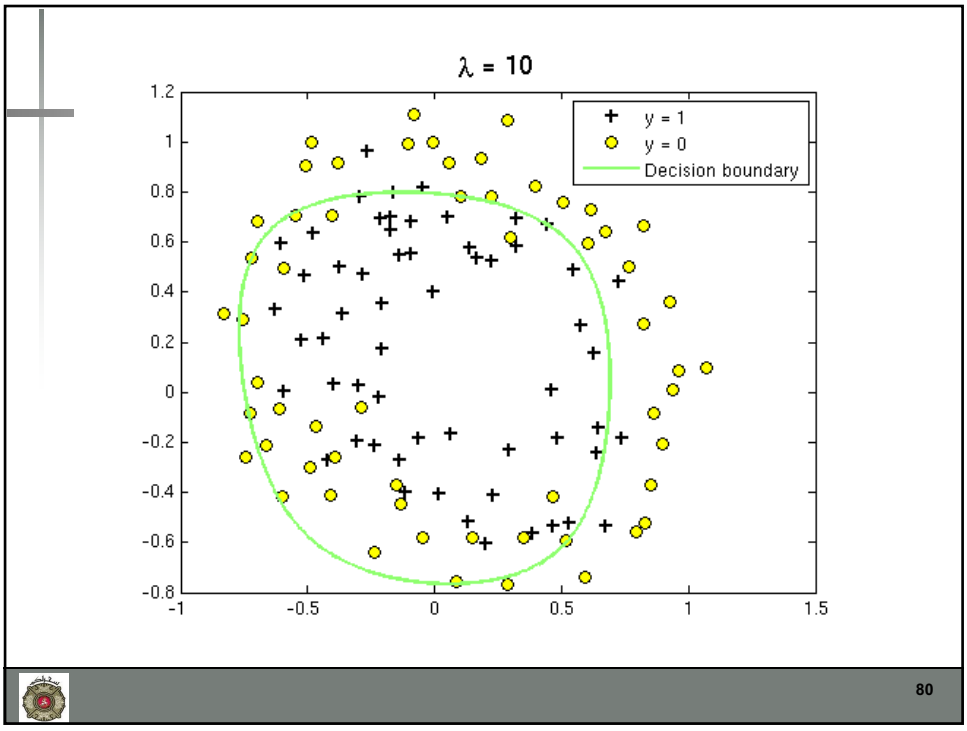


78

78


λ = 1
y = 1
y = 0
Decision boundary
1.2
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
-0.5
0
0.5
1
1.5
79

79


λ = 10
y = 1
y = 0
Decision boundary
1.2
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
-0.5
0
0.5
1
1.5
80

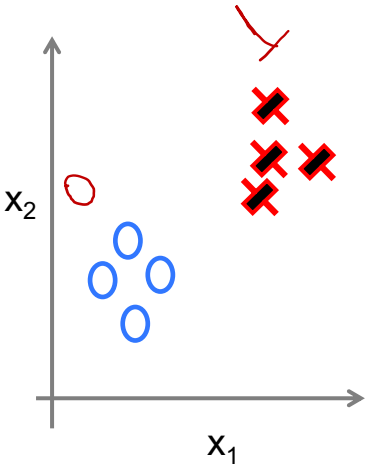
80

MULTI-CLASS CLASSIFICATION

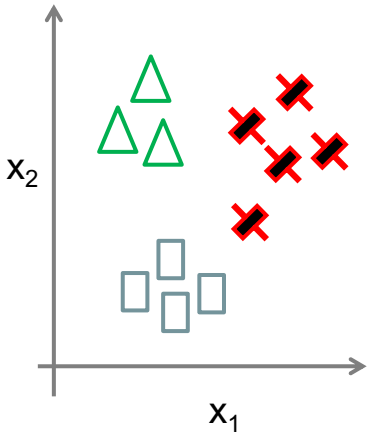
81

81

Binary classification:

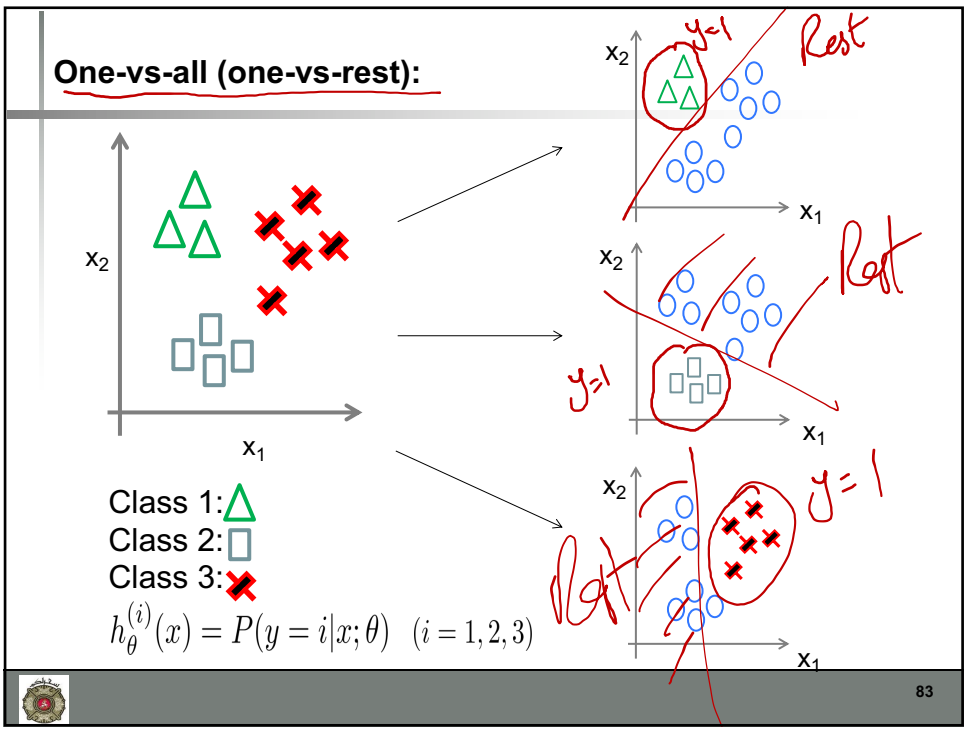


Multi-class classification:



82

82



83

One-vs-all

Train a logistic regression classifier $h_\theta^{(i)}(x)$ for each class i to predict the probability that $y = i$.

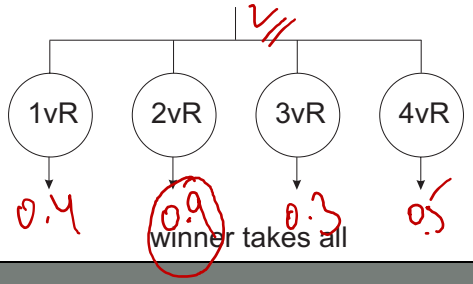
On a new input x , to make a prediction, pick the class i that maximizes

$$\max_i h_\theta^{(i)}(x)$$

84

One vs Rest

- jth classifier is trained with all the training data in the jth class given +ve labels, and the rest given –ve labels.
- Each node is a classifier
- Class of x_i = argmax J(x_i)



85

85

One vs Rest

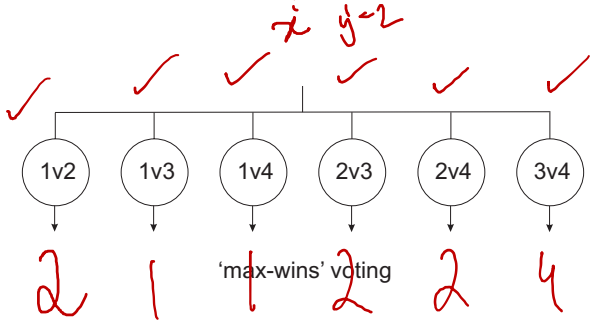
- Benefit:
  - only train M classifiers
- Drawback:
  - Deal with unbalanced data
  - Down sampling or upsampling

86

86

One vs One

- Max-wins voting
- Construct M(M-1)/2 binary classifiers
- Each classifier votes for the class

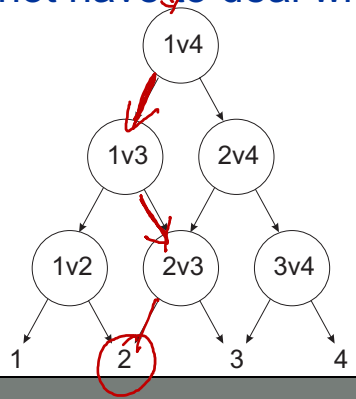


87

87

Directed Acyclic Graph

- Hierarchical classifier
- Construct M(M-1)/2 binary classifiers
- Do not have to deal with unbalanced data

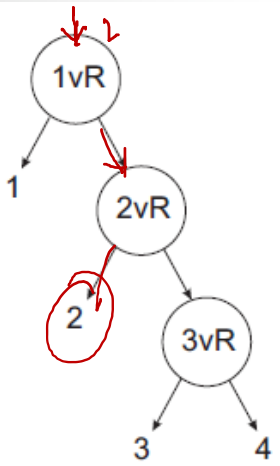


88

88

UnBalanced Decision Tree

- Re-arranged version of 1VR
- Better handling of Imbalanced data



89

89

ACTIVATION FUNCTIONS

90

90

Activation Function

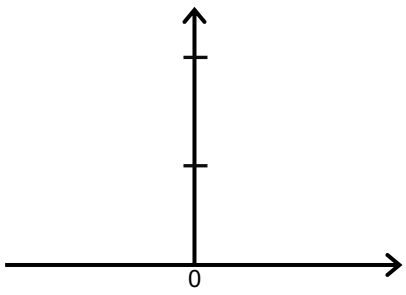
- $h_\theta(x) = \theta^T x$
- Pass this output through a limiting function called Activation function
- $g\big(h_\theta(x)\big) = g(\theta^T x)$
  - Where g is the activation function

91

91

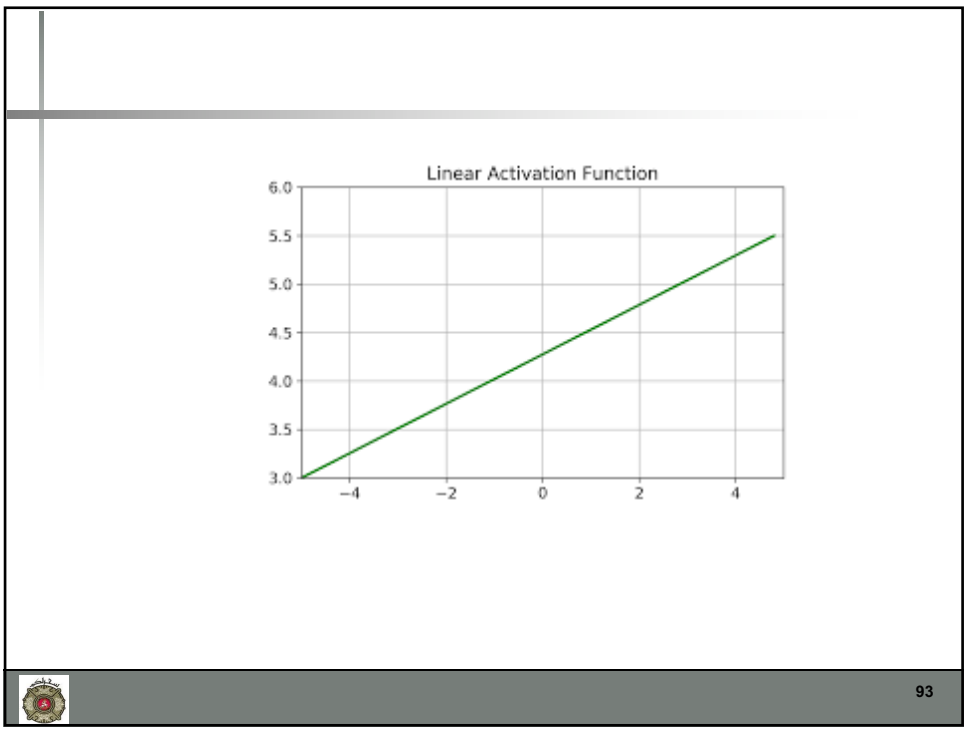
Linear Activation Function

- $g(\theta^T x) = \theta^T x$
- No change to the output



92

92



93

Linear Activation Function

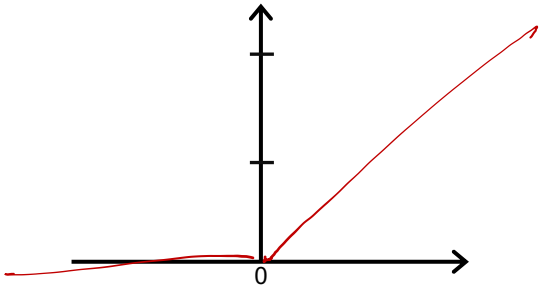
- Similar to the equation of a straight line
- Gives a range of activations from -inf to +inf
- Best suited to for simple regression problems, maybe housing price prediction
- Demerits
  - The derivative of the linear function is the constant

94

94

Rectified Linear Activation(ReLU)

- $g(\theta^T x) = reclin\ (\theta^T x)$
- $g(\theta^T x) = \max(0, \theta^T x)$

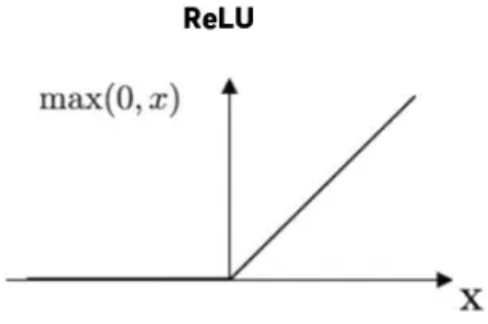


95

95

Rectified Linear Unit (ReLU)

- The formula is pretty simple,
  - if the input is a positive value, then that value is returned otherwise 0

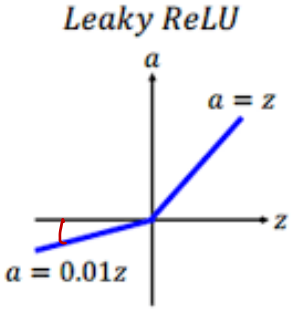


96

96

LeakyReLU

- LeakyReLU is a slight variation of ReLU.
- For positive values, it is same as ReLU, returns the same input,
- For other values, a constant 0.01 with input is provided.

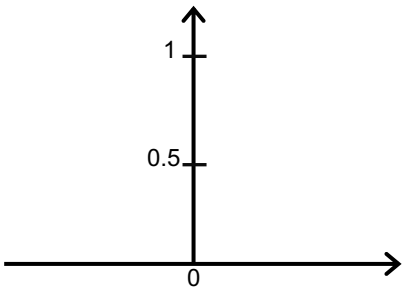


97

97

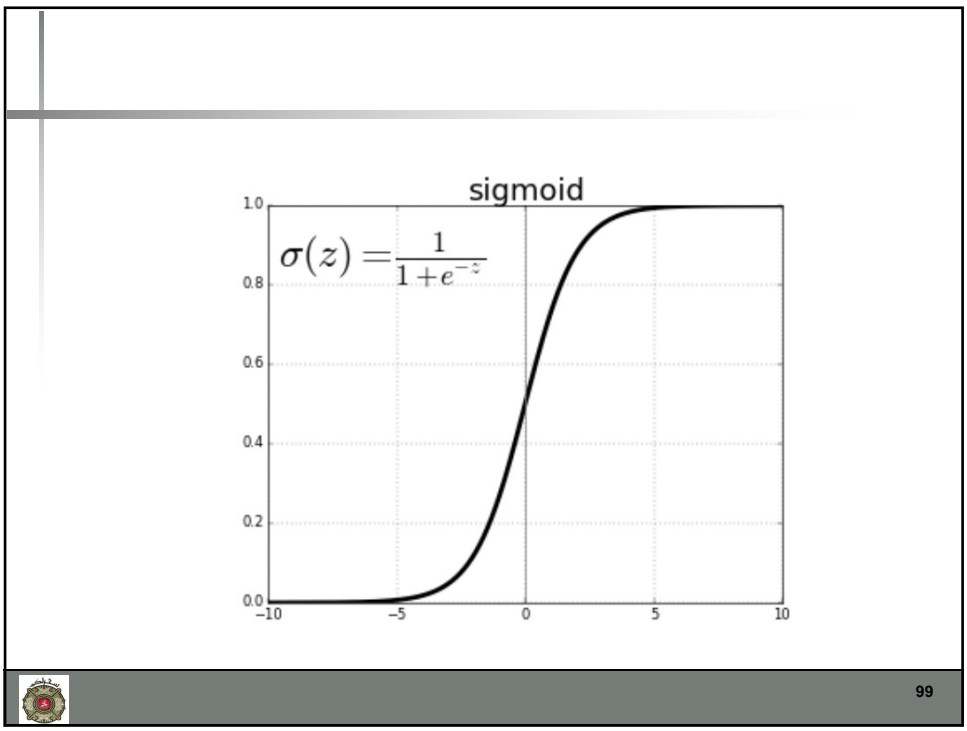
Sigmoid Activation

- $g(\theta^T x) = sigm(\theta^T x)$
- $= \frac{1}{1+e^{(-\theta^T x)}}$
- $= \frac{1}{1+e^{-z}}$

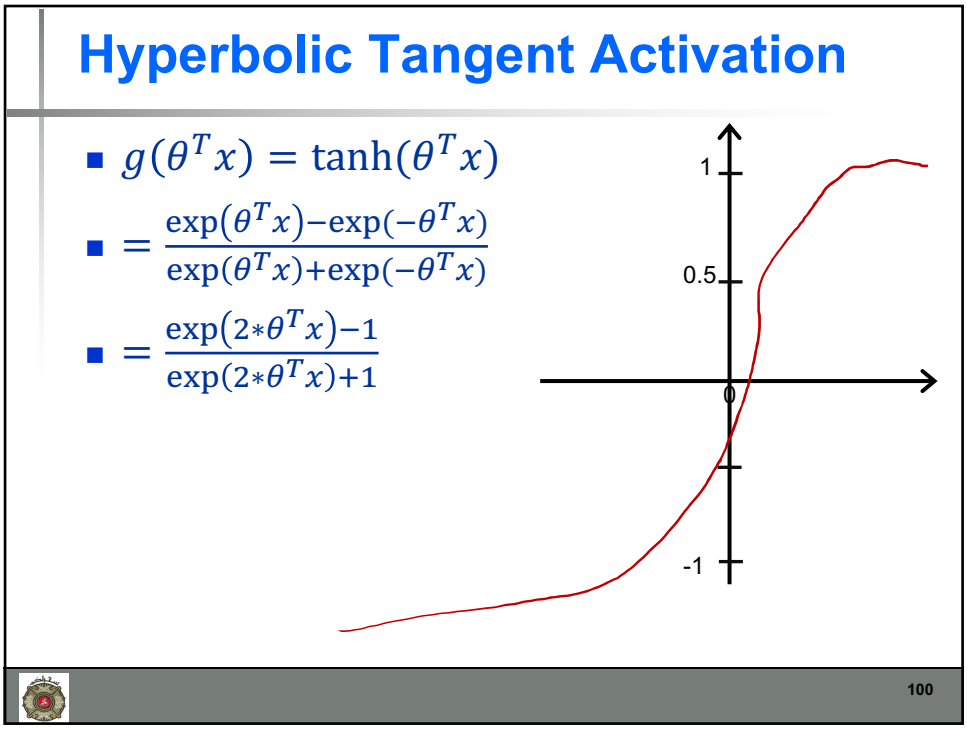


98

98


sigmoid
$\sigma(z) = \frac{1}{1+e^{-z}}$
1.0
0.8
0.6
0.4
0.2
0.0
−10
−5
0
5
10
99

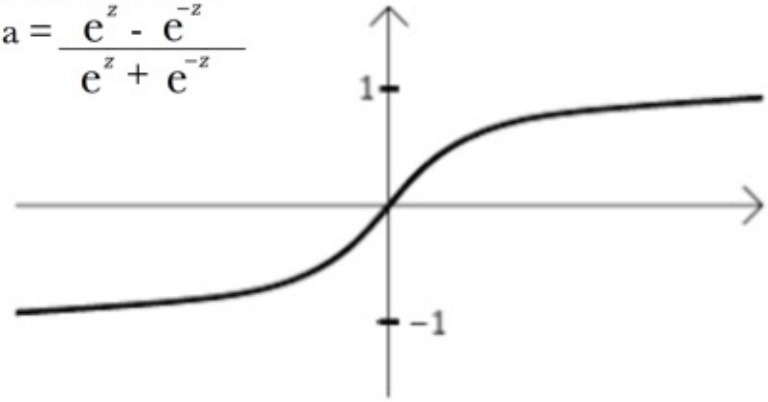
99


Hyperbolic Tangent Activation
$g(\theta^T x) = \tanh(\theta^T x)$
$= \frac{\exp(\theta^T x) - \exp(-\theta^T x)}{\exp(\theta^T x) + \exp(-\theta^T x)}$
$= \frac{\exp(2 * \theta^T x) - 1}{\exp(2 * \theta^T x) + 1}$
1
0.5
0
-1
100

100

Tanh Function

$$a = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$



101

101

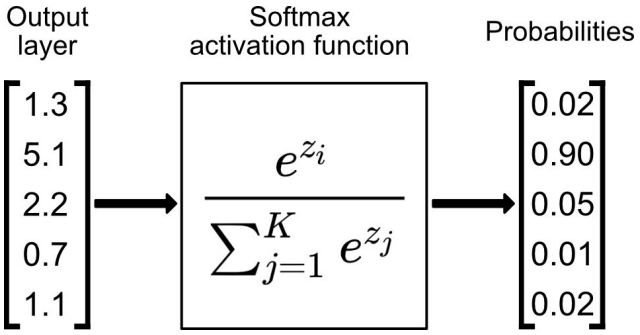
SoftMax

- Softmax activation function returns probabilities of the inputs as output.
- The probabilities will be used to find out the target class.
- Final output will be the one with the highest probability.
- The sum of all these probabilities must be equal to 1.
- This is mostly used in classification problems, preferably in multiclass classification.

102

102

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$



103

103

Steps for Soft Max

1. Exponentiate every element of the output layer and sum the results (around 181.73 in this case)
2. Take each element of the output layer, exponentiate it and divide by the sum obtained in step 1 (exp(1.3) / 181.37 = 3.67 / 181.37 = 0.02)

104

104

Announcements

- Reading Assignment
  - Logistic Regression
  - Reading: Chapter 8
    - Kevin Murphy
  - Wikipedia
    - https://en.wikipedia.org/wiki/Logistic_regression
  - Youtube
    - Video 7: Logistic Regression - Introduction
    - https://youtu.be/gNhogKJ_q7U

105

105

Next Lecture

- Feature Selection
- Feature Reduction

106

106